

Campus Recruitment Command Center

An Integrated System for Student Placements & Corporate Relations

PAWAN KUMAR RAJAK

Table of Contents

Problem Statement.....	1
Phase 1: Problem Understanding & Industry Analysis.....	3
Phase 2: Org Setup & Configuration Summary	6
Phase 3: Data Modeling & Implementation Report.....	9
Phase 4: Process Automation Implementation Final Report	14
Phase 5: Apex Programming Implementation Report	23
Phase 6: User Interface Development Final Report.....	27
Phase 7: Integration & External Access Analysis Report.....	33
Phase 8: Data Management & Deployment Final Report.....	36
Phase 9: Reporting, Dashboards & Security Final Report	40
Phase 10: Final Presentation & Handoff Report	45

Problem Statement

Project Title: "Campus Recruitment Command Center - An Integrated System for Student Placements & Corporate Relations"

Industry: Higher Education

Project Type: B2S (Business-to-Student) & B2B (Business-to-Business) Salesforce Platform

Target Users: Training & Placement (T&P) Officers, Students, and the Head of Placements

Problem Statement

A university's Training & Placement (T&P) cell manages hundreds of corporate relationships and thousands of student applications using disconnected tools like spreadsheets, Google Forms, and email. This manual process results in communication gaps, missed application deadlines for students, and an inability for T&P officers to efficiently match the right

students with the right opportunities. Furthermore, the constant influx of student queries via email overwhelms the T&P staff, leading to slow response times for critical questions.

To address this, the university wants to implement a Salesforce platform to:

- Centralize company and job posting information.
- Automate the student application and notification process.
- Streamline and automate the resolution of common student queries.
- Provide real-time dashboards for the Head of Placements to track success metrics.

Use Cases

1. Corporate & Recruiter Management

- Maintain a unified database of all visiting companies and their recruiter contact information.
- Track the status of engagement with new companies (e.g., Approached, MoU Signed, Visit Scheduled).
- Log all interactions, such as calls and meetings, with company representatives.

2. Job Posting & Application Management

- Create and manage detailed Job Postings with specific eligibility criteria (e.g., Branch, GPA, Graduation Year).
- Automatically notify eligible students via email when a new relevant job is posted.
- Allow students to apply for jobs through a simple, guided interface (a Screen Flow).
- Track the status of each student's application (e.g., Applied, Shortlisted, Offer Extended).

3. Automated Student Query Resolution

- Automatically create a "Query" record in Salesforce when a student sends an email to a designated placement helpdesk address (e.g., placements@university.edu).
- Send an automated initial email response acknowledging receipt of the query.
- Use a Flow to scan the query's subject for keywords (e.g., "Resume," "Eligibility," "Deadline") and send back a templated email with helpful links or standard answers, closing common queries without manual intervention.

4. Reporting & Analytics

- A master dashboard for the Head of Placements showing key metrics like:

- Number of Companies Visiting
- Total Job Openings vs. Applications
- Branch-wise Placement Percentage
- Reports that track the placement funnel and T&P officer activity.

Phase 1: Problem Understanding & Industry Analysis

Date: September 10, 2025

1. Requirement Gathering

The primary goal of this project is to transform the university's manual and fragmented placement process into a streamlined, automated, and centralized system using the Salesforce platform.

Functional Requirements (What the system must do):

- **Company & Recruiter Management:** A central database to store and manage information about visiting companies (Accounts) and their recruiters (Contacts).
- **Job Posting Management:** T&P officers must be able to create, update, and manage job postings with specific eligibility criteria (e.g., Branch, GPA, required skills).
- **Automated Student Notifications:** The system must automatically email eligible students as soon as a new, relevant job is posted.
- **Student Application Portal:** Students need a simple, guided way to view and apply for open positions.
- **Application Tracking:** The system must track the status of each student's application (Applied, Shortlisted, Interviewing, Offer Extended, Rejected).
- **Automated Query Handling:** The system must automatically create a case/record from incoming student emails and provide an automated, templated response for common queries.
- **Reporting & Analytics:** The Head of Placements requires real-time dashboards to view key metrics like placement rates, company engagement, and application numbers.

Technical Requirements:

- The solution must be built on the Salesforce Lightning Platform.
- The system must enforce data security through a role-based model (T&P Officer vs. Student).

- The solution should be scalable to handle data for thousands of students and hundreds of companies.

2. Stakeholder Analysis

The success of the project depends on meeting the needs of its key users.

Stakeholder Role	Needs & Goals	Current Pain Points
T&P Officer	- Quickly add new companies and jobs. Reduce manual data entry and email tasks. Track all student applications in one place.	- Data is scattered across multiple spreadsheets. Manually filtering and emailing students is time-consuming and error-prone. Overwhelmed by repetitive student questions.
Student	- A single, reliable source for all job openings. Timely alerts for jobs they are eligible for. A clear and simple application process.	- Fear of missing deadlines due to communication gaps. Confusion from information on different platforms (email, WhatsApp, website). Slow response times for important queries.
Head of Placements	- High-level, real-time view of placement statistics. Data-driven insights to improve corporate relations. Professional reports for university management.	- Lack of accurate, real-time data for strategic decision-making. Reporting is a manual and time-consuming process at the end of the placement season.

3. Business Process Mapping

As-Is Process (Current State):

1. A company recruiter emails job details to a T&P Officer.
2. The T&P Officer manually checks a master student spreadsheet to find eligible candidates.

3. The Officer composes and sends a bulk email to the filtered list.
4. Students apply by replying to the email with their resumes.
5. The T&P Officer tracks these applications in another spreadsheet.
6. This process is slow, inefficient, and creates multiple versions of the truth.

To-Be Process (Future State with Salesforce):

1. A T&P Officer creates a Job Posting record in Salesforce with defined eligibility criteria.
2. **SYSTEM ACTION:** A Record-Triggered Flow instantly sends a templated email to all Contact records (Students) who meet the criteria.
3. The student clicks a link in the email, launching a Screen Flow to apply.
4. **SYSTEM ACTION:** Upon flow completion, an Application record is automatically created, linking the Student and the Job Posting.
5. All data is instantly available on a real-time dashboard for the Head of Placements.

4. Industry-specific Use Case Analysis (Higher Education)

In the Higher Education sector, the Training & Placement cell is a critical function that directly impacts the university's reputation and ranking. This project addresses a core industry challenge: bridging the gap between academia and the corporate world effectively.

- **Enhancing the Student Experience:** A smooth, transparent, and professional placement process is a key differentiator for universities. This system directly enhances the student experience at the most critical final stage of their journey.
- **Strengthening Corporate Partnerships:** Providing corporate partners with a streamlined recruiting process makes the university a more attractive place to hire from, leading to better and more numerous job opportunities for students.
- **Data-Driven Strategy:** This system moves the T&P cell from a purely administrative function to a strategic one, allowing the Head of Placements to use data to identify trends and improve their outreach strategies.

5. AppExchange Exploration

A review of the Salesforce AppExchange was conducted to identify existing solutions.

- **Salesforce Education Cloud:** This is Salesforce's flagship product for the education industry. It is an extensive, enterprise-level solution that manages the entire student lifecycle, from admissions to alumni relations. While powerful, it is too large and complex for the specific, immediate needs of the T&P cell and the 5-day project scope.

- **Third-Party Apps:** Various apps exist for event management, email marketing, and form building.

Conclusion: A solution that is perfectly tailored to the university's unique placement process, can be developed rapidly, and avoids the cost and complexity of a large, pre-built package. This project will deliver maximum value by focusing directly on solving the T&P cell's most pressing problems.

Phase 2: Org Setup & Configuration Summary

Date: September 11, 2025

1. Core Company Configuration

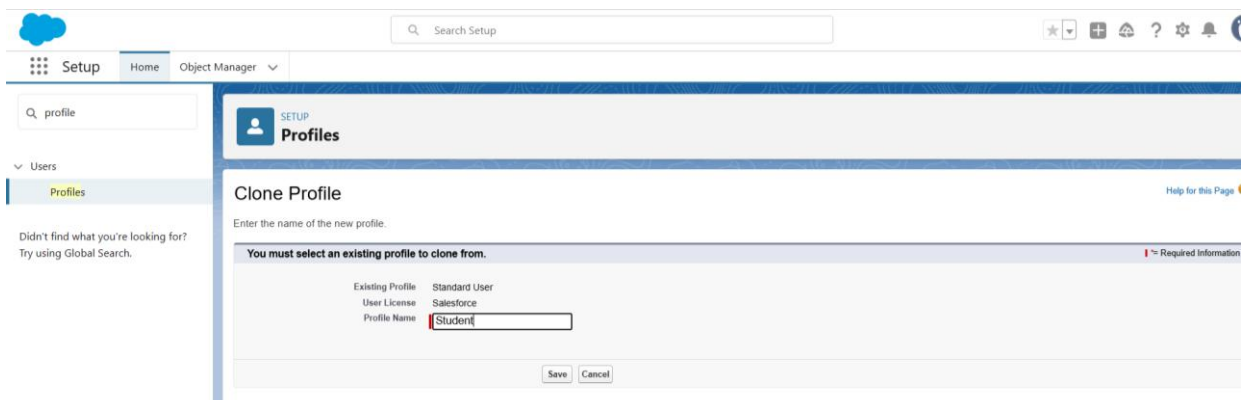
The foundational settings for the Salesforce org were established to mirror the operational environment of the university's T&P Cell.

- **Company Information:** The org was configured with the university's name, default time zone (Asia/Kolkata), and locale to ensure all time-stamped data is accurate.
- **Business Hours & Fiscal Year:** Standard business hours (Mon-SAT, 12 AM - 12 AM) were set to enable future tracking of case/query response times. A standard fiscal year was confirmed for reporting consistency.
- **Developer Org:** A new Salesforce Developer Edition org was created to serve as the dedicated environment for this project.

2. Security Model Implementation

A multi-layered security model was configured to ensure data is accessible only to the appropriate users.

- **Profiles:** Two custom profiles were created by cloning the "Standard User" profile:
 - T&P Officer: For staff who will manage the system.
 - Student: For end-users who will interact with the system.



The screenshot shows the Salesforce Setup interface. The left sidebar has a search bar with 'profile' entered and a list of items including 'Users' and 'Profiles'. The main content area is titled 'SETUP Profiles' and shows a 'Clone Profile' dialog box. The dialog box has a header 'Clone Profile' and a sub-header 'Enter the name of the new profile.' Below this, there is a message 'You must select an existing profile to clone from.' and a table with two columns: 'Existing Profile' and 'User License'. The table has one row with 'Standard User' and 'Salesforce'. Below the table, there is a text input field for 'Profile Name' with the value 'Student' entered. At the bottom of the dialog box, there are 'Save' and 'Cancel' buttons. The 'Save' button is highlighted.

Setup Home Object Manager

Q profiles

Users

Profiles

Didn't find what you're looking for? Try using Global Search.

SETUP Profiles

Profiles

All Profiles Edit Delete Create New View

New Profile

Action	Profile Name	User License
Edit Del ...	Student	Salesforce
Edit Clone	System Administrator	Salesforce
Edit Del ...	T&P Officer	Salesforce
Edit Clone	Work.com Only User	Work.com Only

1-45 of 45 0 Selected Previous Next Page 1 of 1

- **Role Hierarchy:** A top-down data visibility structure was created: **CEO -> Head of Placements -> T&P Officer**. This ensures management can view the records of their subordinates.

Setup Home Object Manager

Q roles

Users

Roles

Feature Settings

Sales

Contact Roles on Contracts

Contact Roles on Opportunities

Service

Case Teams

Case Team Roles

Contact Roles on Cases

Didn't find what you're looking for? Try using Global Search.

SETUP Roles

Creating the Role Hierarchy

You can build on the existing role hierarchy shown on this page. To insert a new role, click **Add Role**.

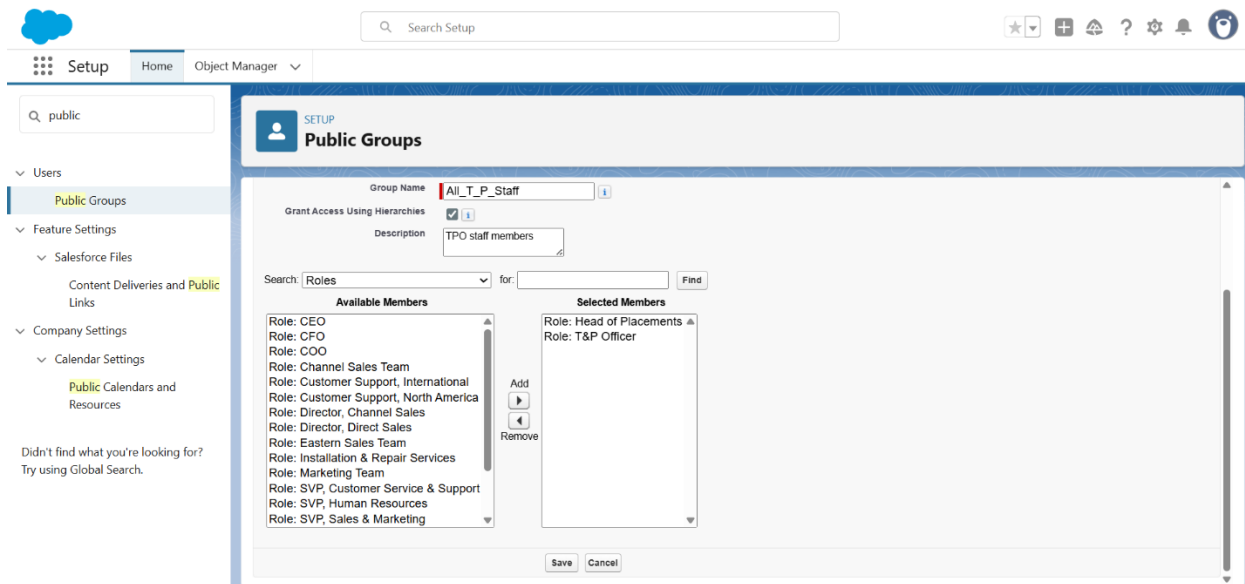
Your Organization's Role Hierarchy

Collapse All Expand All

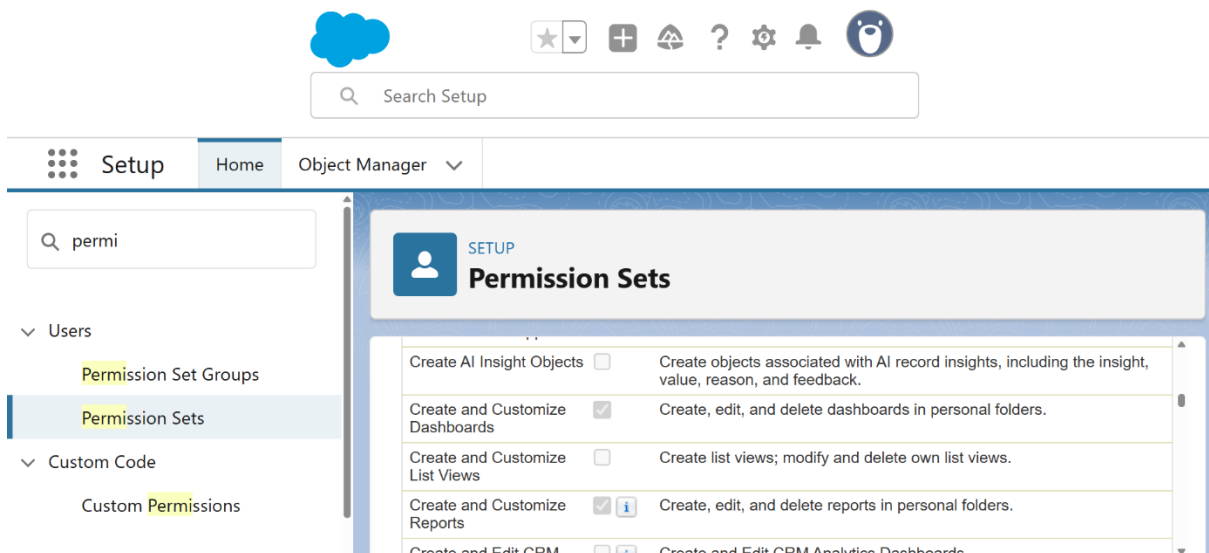
- Gyan Ganga Institute of Technology and Sciences
 - Add Role
 - CEO Edit | Del | Assign
 - Add Role
 - CFO Edit | Del | Assign
 - Add Role
 - COO Edit | Del | Assign
 - Add Role
 - Head of Placements Edit | Del | Assign
 - Add Role
 - T&P Officer Edit | Del | Assign
 - Add Role
 - SVP_Customer Service & Support Edit | Del | Assign
 - Add Role
 - SVP_Human Resources Edit | Del | Assign
 - Add Role
 - SVP_Sales & Marketing Edit | Del | Assign
 - Add Role

Show in tree view

- **Organization-Wide Defaults (OWD):** A **Private** data model was established as the baseline. This is a security best practice, ensuring users can only see their own records by default unless explicitly granted access.



- Permission Set:** A Permission Set named T&P Analytics Manager was created to grant specific, additive permissions (e.g., "Create and Customize Dashboards") to select users without changing their entire profile. This demonstrates a flexible approach to security.



3. Deferred Configurations & Justification

Certain configurations mentioned in the project checklist were intentionally deferred to a later phase for strategic reasons.

- **Sharing Rules:** Sharing Rules are used to open up access to records that are locked down by a Private OWD. These rules are record-specific. **Reason for Deferral:** The custom objects (Job Posting, Application, T&P Query) have not been created yet. Sharing Rules for these objects will be configured in Phase 4 after the data model is built.
- **Login IP Ranges:** This security feature restricts user logins to specific IP addresses. **Reason for Deferral:** During the development phase, it is impractical to restrict IP ranges as development may occur from various locations. This is a production-level security measure that would be implemented post-development, before final deployment.
- **Sandbox Usage & Deployment:** These concepts relate to the development lifecycle in a team environment. **Reason for Deferral:** As a rapid, 5-day solo project, development is being done directly in a Developer Org for maximum efficiency. In a real-world team project, a sandbox-driven development and deployment strategy would be utilized.

Phase 3: Data Modeling & Implementation Report

Date: September 12, 2025

1. Overview

This document outlines the actions taken during the Data Modeling phase of the project. The objective was to create a robust, scalable, and secure data structure to support all functionalities of the Training & Placement (T&P) application. This involved configuring standard Salesforce objects, creating new custom objects and fields, establishing relationships, and enhancing the user interface.

2. Standard Object Configuration

Standard Salesforce objects were leveraged and configured to fit the project's specific terminology and data segregation needs.

- **Account Object:**

- **Action:** The Account object was renamed to Company.
- **Justification:** This provides a more intuitive user experience for T&P Officers, who primarily manage relationships with companies, not "accounts."
- **Contact Object (with Record Types):**
 - **Action:** Two record types were created on the Contact object: Student and Recruiter. Each record type was assigned a unique page layout (Student Layout and Recruiter Layout).
 - **Justification:** The Contact object stores two distinct types of people. Record Types are the Salesforce best practice for segregating them, allowing for different data fields and user experiences on the same object.
- **Lead Object:**
 - **Action:** The standard Lead object was not renamed or configured for this project's initial scope.
 - **Justification:** The primary focus is on managing relationships with established corporate partners (Companies) and student applications. While future enhancements could use Leads for tracking potential new corporate partners, it is not part of the core requirement for this 5-day build.

3. Custom Object & Field Creation

Three core custom objects were created to form the backbone of the application.

Object 1: Job Posting (Job_Posting__c)

Object 1: Job Posting (Job_Posting__c)

Field Label	API Name	Data Type	Details
Job Posting Name	Name	Text	The primary identifier/title of the job.
Company	Company__c	Master-Detail	A required link to the parent Company (Account) record.
Description	Description__c	Text Area (Long)	For the detailed job description.
Location	Location__c	Text	The city/location of the job.
Package (CTC)	Package_CTC__c	Currency	The offered Cost-to-Company.

Field Label	API Name	Data Type	Details
Eligibility Branch	Eligibility_Branch__c	Picklist (Multiselect)	Values: Computer Science, Mechanical, Electronics, Civil.
Minimum GPA	Minimum_GPA__c	Number (2,1)	Required GPA for eligibility.
Status	Status__c	Picklist	Values: Open, Interviewing, Closed.

Object 2: Application (Application__c) – Junction Object

Field Label	API Name	Data Type	Details
Application ID	Name	Auto-Number	Format: APP-{0000}.
Job Posting	Job_Posting__c	Master-Detail	A required link to the parent Job_Posting__c record.
Student	Student__c	Master-Detail	A required link to the parent Contact (Student) record.
Status	Status__c	Picklist	Values: Applied, Shortlisted, Interview Scheduled, Offer Extended, Rejected.

Object 3: T&P Query (T_P_Query__c)

Field Label	API Name	Data Type	Details
Query ID	Name	Auto-Number	Format: Q-{00000}.
Student	Student__c	Lookup	An optional link to the Contact (Student) record.
Description	Description__c	Text Area (Long)	The full text of the student's question.
Status	Status__c	Picklist	Values: New, Automated Reply Sent, Manual Follow-up Needed, Closed.

Field Label	API Name	Data Type	Details
Category	Category__c	Picklist	Values: Eligibility, Resume, Deadline, Technical Issue, Other.

4. Relationship Decisions (Master-Detail vs. Lookup)

The choice of relationship type is critical for data integrity.

- **Master-Detail Relationship (MDR):** Used on the Application object for its links to Job Posting and Contact.
 - **Justification:** An application is fundamentally dependent on both a job and a student. It cannot exist on its own. The MDR enforces this by ensuring an application record is automatically deleted if either its associated job or student is deleted (cascading delete). This also allows for rollup summary fields in the future.
- **Lookup Relationship:** Used on the T&P Query object to link to the Contact.
 - **Justification:** A query is related to a student, but it is also a historical record for the T&P cell. If a student's record is deleted after graduation, the T&P cell may still want to retain the query for analytics. A Lookup provides a looser connection that prevents the query from being auto-deleted, thus preserving historical data.

5. User Interface & Field-Level Security

- **Page Layouts & Compact Layouts:** Custom page layouts were created for the Student and Recruiter record types. A custom Compact Layout, Job Posting Highlights, was created to show key information at the top of the job posting page.
- **Field-Level Security (FLS):** FLS was configured to ensure users only see relevant data.
 - **User-Driven Modification:** Based on user requirements for transparency, the Package (CTC)__c field on the Job Posting object has been made **Read-Only** for the Student profile. This allows students to make informed decisions while preventing them from editing the data.
 - **Internal Fields:** The Category__c field on the T&P Query object was **hidden** from the Student profile as it is an internal classification field for T&P officers only.

- Q profile

Users

Profiles

Didn't find what you're looking for?
 Try using Global Search.

SETUP

Profiles

Data Share Sagemaker Connections	☐	☐	☐	☐	☐	☐	Work Plan Templates	☒	☒	☒	☐	☒	☐
Data Share Snowflake Connections	☐	☐	☐	☐	☐	☐	Work Step Templates	☒	☒	☒	☐	☒	☐
Data Share Targets	☐	☐	☐	☐	☐	☐	Work Types	☒	☒	☐	☐	☐	☐
Data Share Target Connection	☐	☐	☐	☐	☐	☐	Work Type Groups	☒	☒	☒	☐	☐	☐

Custom Object Permissions

	Basic Access				Data Administration			
	Read	Create	Edit	Delete	View All Records	Modify All Records	View All Fields	
Applications	☒	☒	☐	☐	☐	☐	☐	
Job Postings	☒	☐	☐	☐	☐	☐	☐	

	Basic Access				Data Administration			
	Read	Create	Edit	Delete	View All Records	Modify All Records	View All Fields	
T&P Queries	☒	☒	☐	☐	☐	☐	☐	

Session Settings

Session Times Out After	2 hours of inactivity	Session Security Level Required at Login
-------------------------	-----------------------	--

Password Policies

User passwords expire in	90 days
Enforce password history	3 passwords remembered
Minimum password length	8
Password complexity requirement	Must include alpha and numeric characters

Phase 4: Process Automation Implementation Final Report

Date: September 17, 2025

1. Overview

This document provides a comprehensive summary of the process automation implemented during Phase 4. Building upon the secure data model from the previous phases, the focus was to embed the T&P Cell's business logic directly into the application. This was achieved by creating a suite of automations that enforce data quality, guide user actions, eliminate manual work, and ensure timely, professional communication. All automations were built using Salesforce Flow, adhering to modern best practices for scalability and maintenance.

2. Implemented Automations & Processes

A. Validation Rule: Preventing Data Inconsistency

A validation rule was implemented to protect data integrity and enforce a critical business process.

- **Business Need:** The T&P Cell cannot officially close a Job Posting if there are still students with active applications for that role. This prevents administrative errors and ensures no student is left in limbo.
- **Implementation Steps:**
 1. A **Roll-Up Summary Field** named Active Application Count was created on the Job Posting object. This field provides a real-time count of all related Application records that are not in a final state (Offer Extended or Rejected).

Setup > OBJECT MANAGER

Job Posting

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Search Layouts

Aggregates

☒ COUNT

☐ SUM

☐ MIN

☐ MAX

Field to Aggregate: --None--

Filter Criteria

☐ All records should be included in the calculation

☒ Only records meeting certain criteria should be included in the calculation

Field	Operator	Value	
Status	not equal to	Offer Extended	AND
Status	not equal to	Rejected	AND
--None--	--None--		AND
--None--	--None--		AND
--None--	--None--		AND

For checkbox fields, enter a value of True for checked or False for not checked. For picklist fields, enter the master picklist field value in your corporate language.

Previous Next Cancel

2. A **Validation Rule** was then created on the Job Posting object. This rule checks two conditions upon saving a record: if the Status is being changed to "Closed" AND if the Active Application Count is greater than 0. If both are true, the save action is blocked, and a clear error message is displayed to the user.

Setup > OBJECT MANAGER

Job Posting

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Search Layouts

List View Button Layout

Restriction Rules

Scoping Rules

Job Posting Validation Rule

Define a validation rule by specifying an error condition and a corresponding error message. The error condition is written as a Boolean formula expression that returns true or false. When the formula expression returns true, the save will be aborted and the error message will be displayed. The user can correct the error and try again.

Validation Rule Edit Save Save & New Cancel

Rule Name: Prevent_Closure_With_Active_Apps

Active: ☒

Description:

Error Condition Formula

Example: `Discount_Percent__c > 0.30` More Examples

Display an error if Discount is more than 30%

If this formula expression is **true**, display the text defined in the Error Message area

Insert Field Insert Operator

```
AND(
  ISPICKVAL(Status__c, "Closed"),
  Active_Application_Count__c > 0
)
```

Functions

-- All Function Categories --

ABS

ACOS

ADDMONTHS

AND

ASCII

ASIN

Insert Selected Function

ABS(number)
Returns the absolute value of a number, a number without its sign

Quick Tips

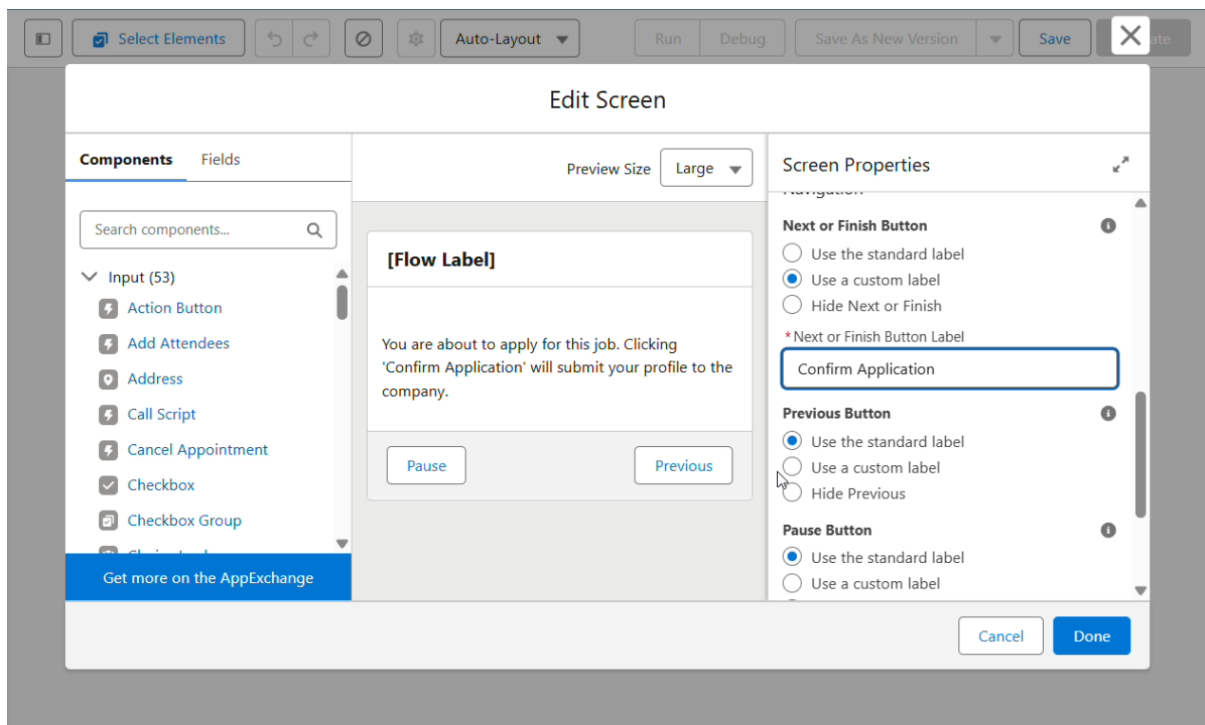
- Operators & Functions

Help for this Page

B. Screen Flow: A Guided Application Experience for Students

To enhance the student user experience and streamline the application process, a guided screen flow was developed.

- **Business Need:** Students require a simple, intuitive, and error-proof method to apply for jobs. A manual process of creating records would be inefficient and confusing.
- **Implementation Steps:**
 1. A **Screen Flow** titled "Student Application Process" was built. It features a simple confirmation screen to ensure the student's intent.
 2. The flow is intelligent, automatically capturing the recordId of the Job Posting it was launched from and the ContactId of the logged-in student user (\$User.ContactId).
 3. Upon confirmation, a Create Records element automatically generates a new Application record, correctly linking the student and job, and setting the initial Status to "Applied".
 4. This flow was then made accessible by creating a **Quick Action** button labeled "Apply Now" and adding it to the Job Posting page layout, ensuring high visibility.



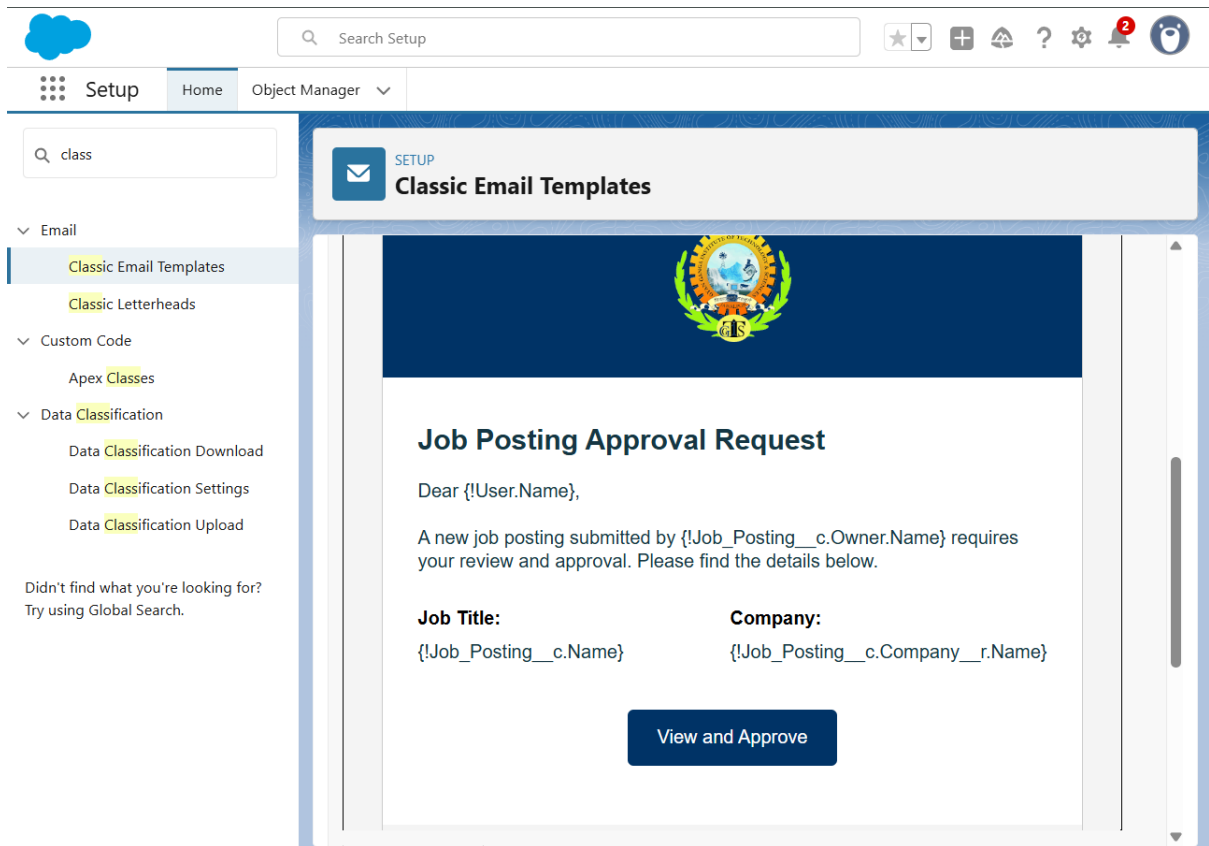
The screenshot displays the Salesforce Flow Builder interface for a process titled 'Student Application Process - V1'. The flow is currently inactive and was last saved on 9/17/2025 at 02:43 PM. The flow diagram on the left shows a sequence of steps: 'Screen Flow Start', 'Application Confirmation Screen', 'Create Application' (highlighted), and 'End'. The 'Create Application' step is a 'Create Records' action. The configuration panel on the right shows the following settings:

- Create Records:** Triggered 'Manually'.
- Create a Record of This Object:** 'Application'.
- Set Field Values for the Application:**
 - Field: 'Job Posting' (Value: 'recordId')
 - Field: 'Student' (Value: 'Running User > ContactId')
 - Field: 'Status' (Value: 'Applied')
- Manually assign variables (advanced):** (unchecked)
- Check for Matching Records:** (unchecked)

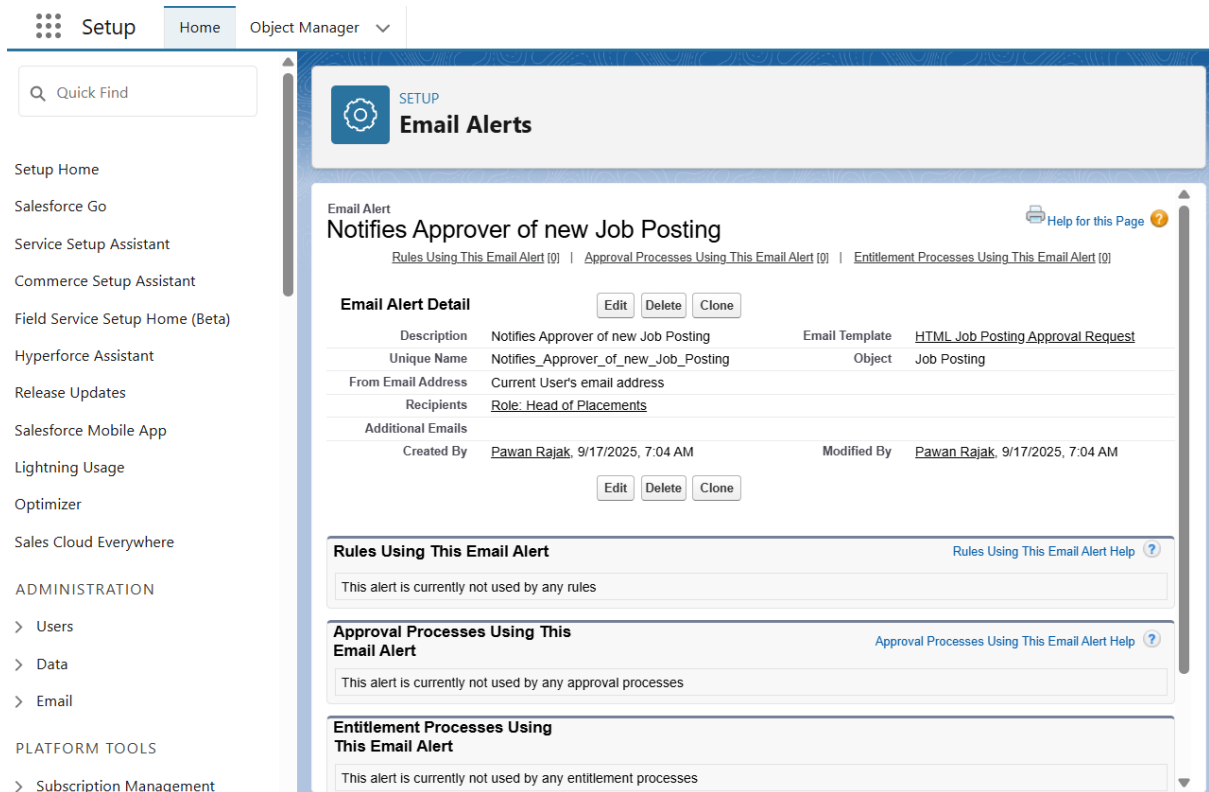
C. Approval Process & HTML Email Alert: Formalizing Job Postings

A formal approval process, supported by a professional HTML email alert, was created to ensure management oversight.

- **Business Need:** All new job postings must be reviewed and approved by the Head of Placements before they are made public to students, ensuring quality and consistency. The notification for this approval must be professional and actionable.
- **Implementation Steps:**
 1. An **HTML Email Template** was created to provide a branded, easy-to-read notification. The template includes the university logo, merges in key details like the Job Title and Company, and features a prominent "View and Approve" button that links directly to the record in Salesforce.



2. An **Email Alert** action was configured to use this HTML template and send it to any user with the Head of Placements role.



3. An **Approval Process** was configured on the Job Posting object. It triggers when a record's Status is set to "Submitted for Approval".

The screenshot shows the Salesforce Setup interface. The left sidebar has a search bar with 'appro' and a list of categories: Data, Feature Settings, and Process Automation. Under 'Process Automation', 'Approval Processes' is selected. The main content area is titled 'Approval Processes' and shows the configuration for 'Job Posting: Job Posting Approval'. The 'Process Definition Detail' section includes fields for Process Name, Unique Name, Description, Entry Criteria, Record Editability, and Approval Assignment. The 'Initial Submitters' field is set to 'Account Owner'. The 'Created By' and 'Modified By' fields are both set to 'Pawan Rajak'.

Process Definition Detail	
Process Name	Job Posting Approval
Unique Name	Job_Posting_Approval
Description	
Entry Criteria	Job Posting: Status EQUALS Submitted for Approval
Record Editability	Administrator ONLY
Approval Assignment	Account Owner
Initial Submitters	Account Owner
Created By	Pawan Rajak, 9/17/2025, 6:15 AM
Modified By	Pawan Rajak, 9/17/2025, 6:15 AM

4. The approval request is automatically assigned to the submitter's manager via the Role Hierarchy.
5. The Email Alert was added to the "Initial Submission Actions," ensuring the approver is notified instantly.

The screenshot shows the 'Initial Submission Actions' configuration for the 'Job Posting: Job Posting Approval' process. The 'Initial Submission Actions' section is highlighted with a red box. It contains a table with columns for Action, Type, and Description. The actions listed are 'Record Lock' and 'Email Alert'. The 'Email Alert' action is highlighted with a red box. Below the 'Initial Submission Actions' section is the 'Approval Steps' section, which contains a table with columns for Action, Step Number, Name, Description, Criteria, Assigned Approver, and Reject Behavior. The first step is 'Step 1' with the assigned approver 'User:TPO officer' and reject behavior 'Final Rejection'.

Action	Type	Description
Record Lock		Lock the record from being edited
Email Alert		Notifies Approver of new Job Posting

Action	Step Number	Name	Description	Criteria	Assigned Approver	Reject Behavior
Step 1	1	Step 1			User:TPO officer	Final Rejection

- Field Updates were configured to automatically change the Status to "Open" upon approval or "Needs Rework" upon rejection.

Approval Processes

Action	Type	Description
	Record Lock	Lock the record from being edited

Approval Steps

Action	Step Number	Name	Description	Criteria	Assigned Approver	Reject Behavior
Show Actions Edit	1	Step 1			User:TPO officer	Final Rejection

Final Approval Actions

Action	Type	Description
Edit	Record Lock	Lock the record from being edited
Edit Remove	Field Update	Update Status to Open

Final Rejection Actions

Action	Type	Description
Edit	Record Lock	Unlock the record for editing
Edit Remove	Field Update	Update Status to Needs Rework

Recall Actions

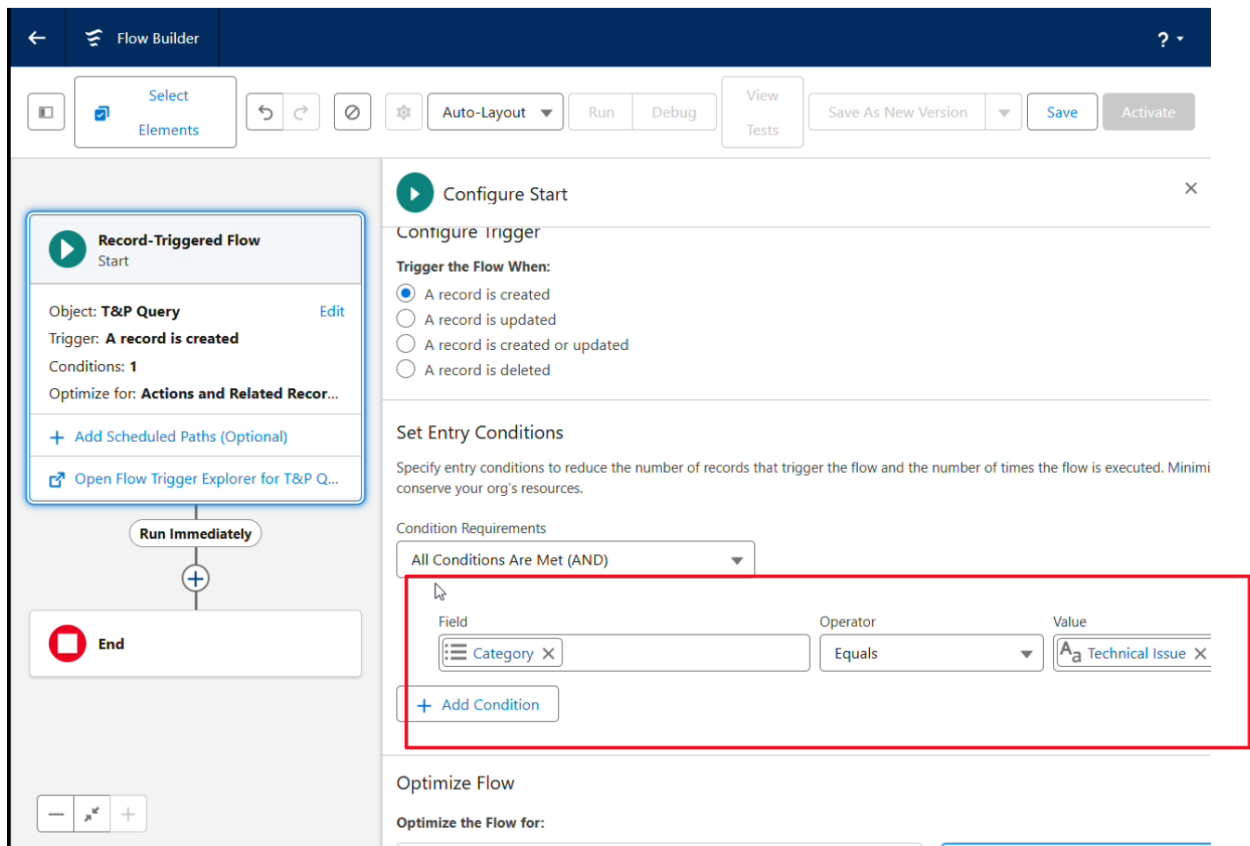
Action	Type	Description
	Record Lock	Unlock the record for editing

[Back To Top](#) Always show me [more](#) records per related list

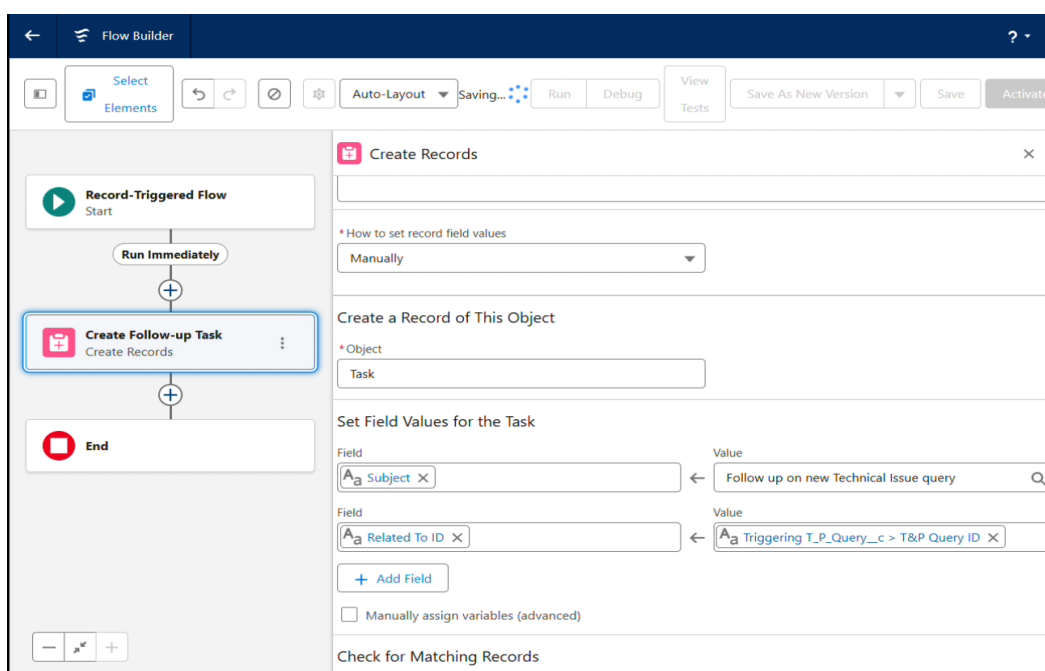
D. Record-Triggered Flow: Automated Task Creation

A background automation was created to ensure that critical student queries are assigned and tracked efficiently.

- Business Need:** Queries related to technical issues need to be flagged and assigned for follow-up so they are not lost in the general queue.
- Implementation Steps (Simplified):**
 - A **Record-Triggered Flow** was built on the T&P Query object that runs only when a new record is created with a Category of "Technical Issue".



2. A Create Records element generates a new Task record.
3. To ensure clarity and trackability, the Task's Subject is set to a static text ("Follow up on new Technical Issue query"), and its Related To ID is dynamically set to the ID of the T&P Query record that triggered the flow (`{!$Record.Id}`). This creates a direct link between the query and the follow-up task.



3. Strategically Excluded Concepts & Justification

- **Legacy Tools (Workflow Rules & Process Builder):** These older automation tools were intentionally not used.
 - **Justification:** Salesforce is actively retiring these tools in favor of Flow Builder. To ensure the application is modern, efficient, and future-proof, all automation was built in Flow. This aligns with industry best practices and simplifies future maintenance.
- **Custom Notifications (In-App Alerts):** The in-app notification feature was deferred for this phase.
 - **Justification:** After analyzing the primary user workflows, it was determined that email is the most critical and reliable communication channel for T&P Officers, who may not be logged into Salesforce at all times. To avoid "notification fatigue" and ensure that high-priority alerts (like an approval request) are never missed, the decision was made to centralize these actionable notifications in the email channel for this initial build. In-app notifications are a candidate for a future enhancement phase, pending user feedback on communication preferences.

Phase 5: Apex Programming Implementation Report

Date: September 19, 2025

1. Overview

This phase involved extending the application's capabilities beyond declarative automation by implementing custom server-side logic with Apex. The primary goal was to enforce a complex business rule that required cross-object data validation, a scenario where an Apex Trigger is the most suitable and efficient tool. The implementation followed Salesforce best practices, including bulkification and the mandatory creation of a comprehensive test class.

2. Business Requirement & Technical Solution

- **Business Need:** To maintain a high quality of student applications and manage company expectations, the T&P Cell must ensure that students only apply for jobs where they meet the minimum academic criteria. Specifically, a student must be prevented from applying for a Job Posting if their GPA is lower than the Minimum GPA required for that role.
- **Chosen Solution:** An **Apex Trigger** on the Application__c object. A standard Validation Rule is not ideal for this scenario as it would require complex cross-object formulas. An Apex Trigger provides the necessary power and flexibility to query related records and enforce the logic cleanly before the application is saved.

3. Implementation Details

A. Prerequisite: GPA Field

To enable the validation, a new custom field was added:

- **Object:** Contact
- **Field Label:** GPA
- **Data Type:** Number(2, 2)
- **Purpose:** To store the official GPA for each student record.

B. Apex Trigger: ApplicationTrigger.trigger

A before insert trigger was developed to run before any new Application__c record is committed to the database.

- **Logic and Best Practices Followed:**
 1. **Bulkification:** The trigger first collects all Job_Posting__c and Student__c IDs from the incoming applications (Trigger.new) into Sets. This best practice

ensures the trigger can process a single record or hundreds of records in one transaction without hitting governor limits.

2. **No SOQL in Loops:** Two efficient SOQL queries are performed *outside* of any loops to retrieve all necessary Job Posting and Contact data at once. The results are stored in Maps for quick and easy data retrieval.
3. **Validation Logic:** The trigger then iterates through the new applications. For each application, it retrieves the corresponding Job Posting and Student records from the Maps. An if statement (Control Statement) then compares the student's GPA__c to the job's Minimum_GPA__c.
4. **Error Handling:** If a student's GPA is too low, the addError() method is called on that specific application record. This action stops the save process for that record and displays a clear, user-friendly error message on the screen.

Full Trigger Code:

```
1  /**
2   * @description Trigger on the Application object to handle custom validation.
3   */
4  trigger ApplicationTrigger on Application__c (before insert) {
5
6      // Best Practice: Create a set to hold the IDs of all Job Postings and Contacts from the new applications.
7      // A Set is a collection that does not allow duplicate values, which is efficient for queries.
8      Set<Id> jobPostingIds = new Set<Id>();
9      Set<Id> studentIds = new Set<Id>();
10
11      // 1. COLLECT IDs: Loop through all the new Application records that are trying to be created.
12      // Trigger.new is a special context variable that holds all the new records in the transaction.
13      for (Application__c app : Trigger.new) {
14          jobPostingIds.add(app.Job_Posting__c);
15          studentIds.add(app.Student__c);
16      }
17
18      // 2. QUERY DATA: Use SOQL to get all the required data in one efficient query for Job Postings
19      // and one for Contacts. This avoids running queries inside a loop, which is a major performance issue.
20      Map<Id, Job_Posting__c> jobPostingsMap = new Map<Id, Job_Posting__c>([
21          SELECT Id, Minimum_GPA__c FROM Job_Posting__c WHERE Id IN :jobPostingIds
22      ]);
23
24      Map<Id, Contact> studentsMap = new Map<Id, Contact>([
25          SELECT Id, GPA__c FROM Contact WHERE Id IN :studentIds
26      ]);
27
28      // 3. VALIDATE LOGIC: Loop through the new applications again. This time, we have all the data we need.
29      for (Application__c app : Trigger.new) {
30          // Retrieve the specific Job Posting and Student for the current application from our Maps.
31          Job_Posting__c relatedJob = jobPostingsMap.get(app.Job_Posting__c);
32          Contact relatedStudent = studentsMap.get(app.Student__c);
33
34          // Control Statement: Check if the student's GPA is less than the job's minimum requirement.
35          // Also check that both values are not null to prevent errors.
36          if (relatedStudent.GPA__c != null &&
37              relatedJob.Minimum_GPA__c != null &&
38              relatedStudent.GPA__c < relatedJob.Minimum_GPA__c) {
39
40              // If the GPA is too low, add a custom error message to the record.
41              // This prevents the record from being saved and displays the error to the user.
42              app.addError('You cannot apply for this job. Your GPA (' + relatedStudent.GPA__c + ') is below the minimum requirement of ' + relatedJob.Minimum_GPA__c + '.');
43          }
44      }
45  }
```

C. Apex Test Class: ApplicationTriggerTest.cls

A dedicated test class was written to validate the trigger's functionality and ensure it is ready for deployment. The class achieved 100% code coverage.

- **Test Setup (@testSetup):** A setup method was used to create all necessary test data in a single transaction. This included a test Company, a Job Posting with a high GPA requirement (9.0), a Student with a qualifying GPA (9.5), and a Student with a non-qualifying GPA (8.0).
- **Positive Test Scenario:** A test method verifies that the studentWithHighGPA can successfully create an Application record. The test passes by using `System.assertEquals()` to confirm that one application record was created.
- **Negative Test Scenario:** A test method verifies that the studentWithLowGPA is blocked from creating an Application. This is tested by wrapping the insert statement in a try-catch block, asserting that the system throws a `DmlException` and that the error message contains the expected text from the trigger.
- Full Test Code:

```

1  /**
2   * @description Test class for the ApplicationTrigger.
3   */
4  @isTest
5  private class ApplicationTriggerTest {
6
7      // @testSetup is a special method that runs once to create all the test data needed by all test methods.
8      @testSetup
9      static void setup() {
10         // Create a test Company
11         Account testCompany = new Account(Name = 'Test Corp');
12         insert testCompany;
13
14         // Create a test Job Posting that requires a high GPA
15         Job_Posting__c testJob = new Job_Posting__c(
16             Name = 'Senior Developer',
17             Company__c = testCompany.Id,
18             Minimum_GPA__c = 9.0
19         );
20         insert testJob;
21
22         // Create two test Students (Contacts)
23         Contact studentWithHighGPA = new Contact(
24             LastName = 'Green',
25             RecordTypeId = Schema.SObjectType.Contact.getRecordTypeInfoByName().get('Student').getRecordTypeId(),
26             GPA__c = 9.5
27         );
28         Contact studentWithLowGPA = new Contact(
29             LastName = 'Red',
30             RecordTypeId = Schema.SObjectType.Contact.getRecordTypeInfoByName().get('Student').getRecordTypeId(),
31             GPA__c = 8.0
32         );
33         insert new List<Contact>{ studentWithHighGPA, studentWithLowGPA };
34     }
35
36     // A positive test method to verify that a student with a high GPA CAN apply.
37     @isTest
38     static void testApplication_PositiveScenario() {
39         // Get the test data we created in the setup method.
40         Job_Posting__c job = [SELECT Id FROM Job_Posting__c LIMIT 1];
41         Contact student = [SELECT Id FROM Contact WHERE LastName = 'Green' LIMIT 1];
42
43         // Start the test. This creates a fresh set of governor limits.
44         Test.startTest();
45
46         // Perform the action: Create an Application record.
47         Application__c app = new Application__c(
48             Job_Posting__c = job.Id,
49             Student__c = student.Id
50         );
51         insert app;
52
53         // Stop the test.
54         Test.stopTest();
55
56         // Assertion: Verify that the application was successfully created by querying it.
57         // If the record exists, the test passes.
58         List<Application__c> createdApps = [SELECT Id FROM Application__c WHERE Id = :app.Id];
59         System.assertEquals(1, createdApps.size(), 'Application should have been created successfully.');
```

4. Strategically Excluded Concepts & Justification

- **Asynchronous Apex (Batch, Queueable, Future, Scheduled):** The business requirement is for immediate, real-time validation at the moment a user tries to save a record. Asynchronous processing, which runs in the background, is not suitable for this use case.
- **Trigger Design Patterns (Handler Classes):** While using a separate handler class is a best practice for complex orgs with multiple triggers on one object, it constitutes over-engineering for this project's scope. The implemented trigger is self-contained and already follows the most critical design principles: bulkification and no DML/SOQL in loops.
- **SOSL (Salesforce Object Search Language):** SOSL is a tool for text-based searches across multiple objects (like a search engine). The trigger's requirement was to fetch specific records based on known IDs, for which SOQL is the correct and more performant tool.

Phase 6: User Interface Development Final Report

Date: September 22, 2025

1. Overview

This phase of the project focused entirely on the User Interface (UI) and User Experience (UX). After building a robust backend with a solid data model, security, and automation, the goal was to create a user-centric front-end that makes the application efficient, intuitive, and pleasant to use. The primary user, the T&P Officer, was at the center of all design decisions.

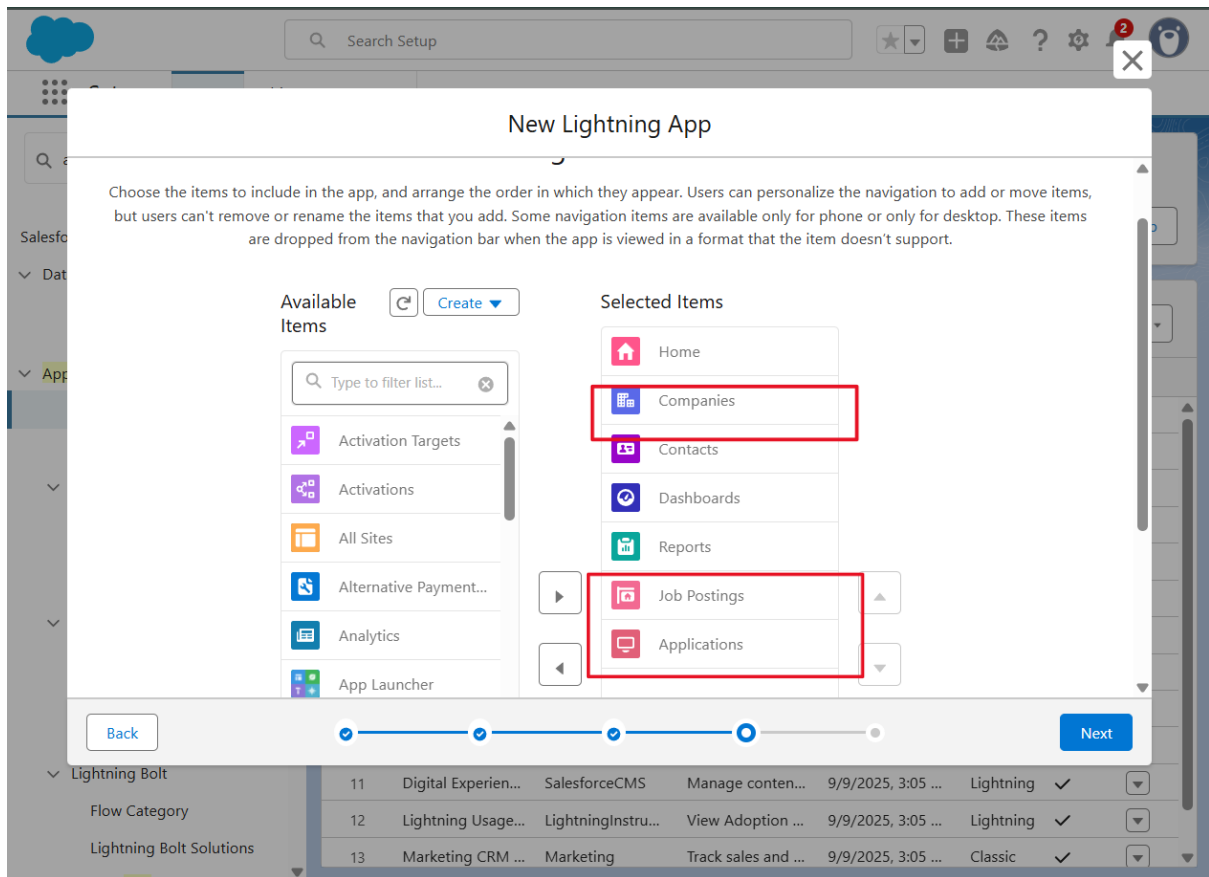
All UI enhancements were achieved using Salesforce's powerful declarative tools, primarily the **Lightning App Builder**. This "clicks-not-code" approach allowed for rapid development and ensures the application is easy to maintain and upgrade in the future.

2. Implemented UI Enhancements & Features

A. Custom Tabs for Custom Objects (Already Created In phase 3)

Before building the main application interface, a foundational step was to create tabs for our custom objects.

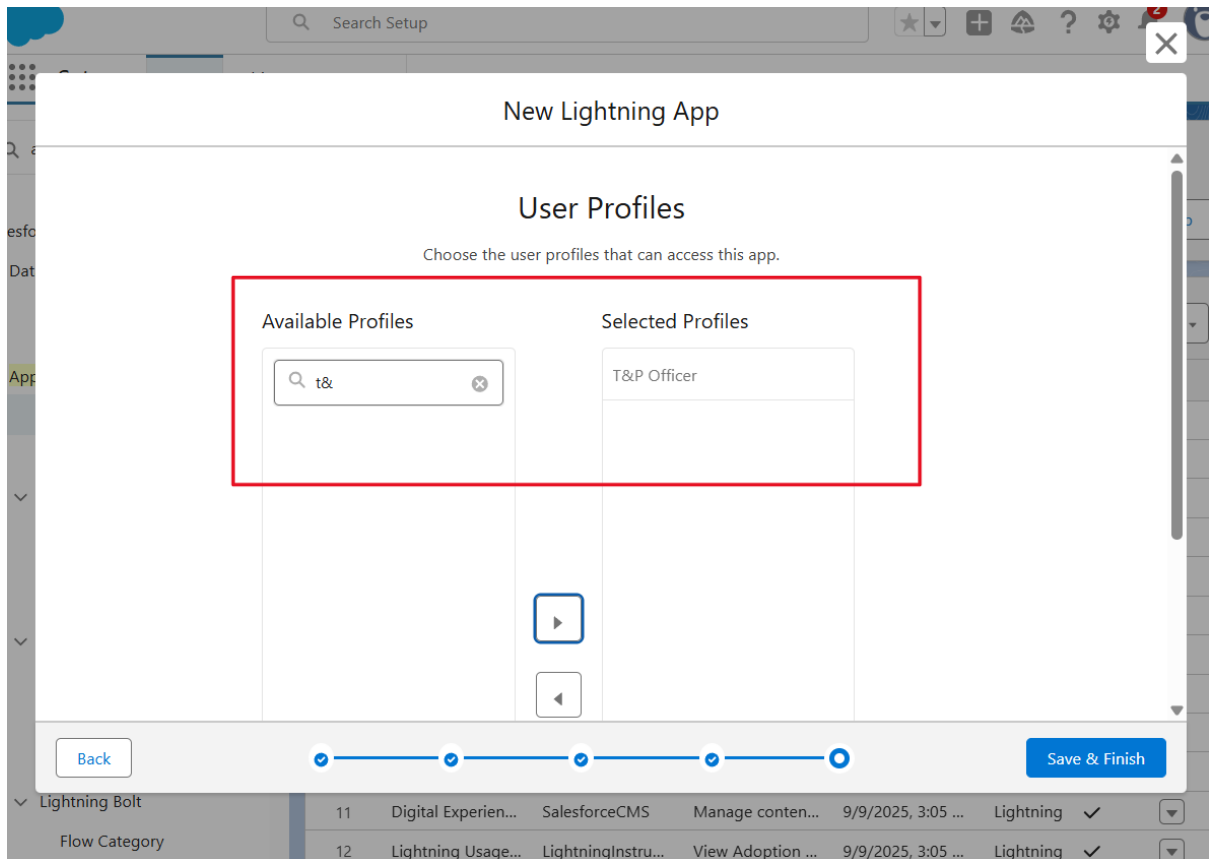
- **Business Need:** In Salesforce, a custom object is just a database table until a Tab is created for it. Without a Tab, users cannot access list views of records or add the object to an app's navigation bar. Creating tabs is the essential first step to making custom objects visible and usable in the UI.
- **Implementation Steps:**
 1. Navigated to Setup > User Interface > Tabs.
 2. In the "Custom Object Tabs" section, a new tab was created for the **Job Posting** object, and a "Briefcase" icon was assigned.
 3. A second new tab was created for the **Application** object, and a "Clipboard" icon was assigned.
 4. During creation, profile visibility was set to "Default On" for the T&P Officer profile and "Tab Hidden" for the Student profile, ensuring only relevant users would see these tabs.



B. Dedicated Lightning App: "Placement Command Center"

A dedicated, branded application was built to serve as the single source of truth for all placement activities.

- **Business Need:** T&P Officers need a focused workspace free from the clutter of standard Salesforce objects like Opportunities or Leads. A custom Lightning App provides a tailored environment, improving navigation, reducing distractions, and reinforcing the T&P Cell's branding.
- **Implementation Steps:**
 1. Using the App Manager in Setup, a new Lightning App was created.
 2. It was named "**Placement Command Center**" and branded with the university's logo and primary color for a professional look and feel.
 3. The navigation was carefully configured to include only the essential tabs in a logical order: Home, Companies, Contacts, Job Postings, Applications, Dashboards, and Reports.
 4. Access to the app was exclusively assigned to the **T&P Officer** user profile, ensuring it remains a dedicated tool for the placement team.

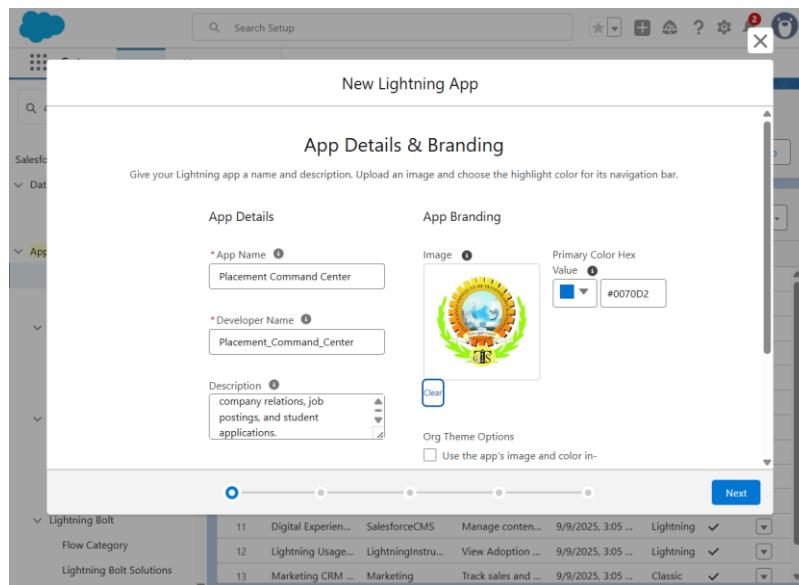


C. Actionable Home Page: "T&P Officer Home Page"

A custom home page was designed to act as a dynamic dashboard, providing T&P Officers with immediate insights the moment they log in.

- **Business Need:** A generic home page does not provide value. The T&P Officer's home page must immediately answer the question, "What do I need to focus on right now?" It should surface pending tasks and key metrics without requiring the user to run reports.
- **Implementation Steps:**
 1. A new **Home Page** was created using the Lightning App Builder, based on the flexible "Header and Three Regions" template.
 2. The page was populated with strategic components:
 - **Rich Text:** To display a clear "Welcome to the Placement Command Center" title.
 - **List View:** Configured to show the "Jobs Submitted for Approval" list view from the Job Posting object. This creates an "Action Items" panel that is always up-to-date.

- **Dashboard Component:** A full dashboard component was added to the main section, which will display key performance indicators once the dashboard is built in Phase 9.
- **Recent Items:** To allow users to quickly navigate back to records they were recently working on.
- This new page was then activated and set as the default home page specifically for the "Placement Command Center" app.



D. Enhanced Record Page: 360-Degree Company View

The standard Company record page was redesigned to provide a comprehensive, "at-a-glance" view of all related activities.

- **Business Need:** When a T&P Officer views a company record, they need to see all associated information—recruiters, open jobs, past applications—without having to click through multiple related lists. An enhanced record page consolidates this information, saving time and improving decision-making.
- **Implementation Steps:**
 1. The default Company Record Page was opened in the Lightning App Builder via the "Edit Page" function.
 2. To maximize the visibility of related data, two **Related List - Single** components were dragged onto the page's sidebar.
 3. The first component was configured to display the **Job Postings** related list for that company.
 4. The second component was configured to display the **Contacts** related list, effectively showing all recruiters from that company.

5. This improved page layout was then activated and assigned as the **Org Default**, ensuring every user benefits from this more efficient view.

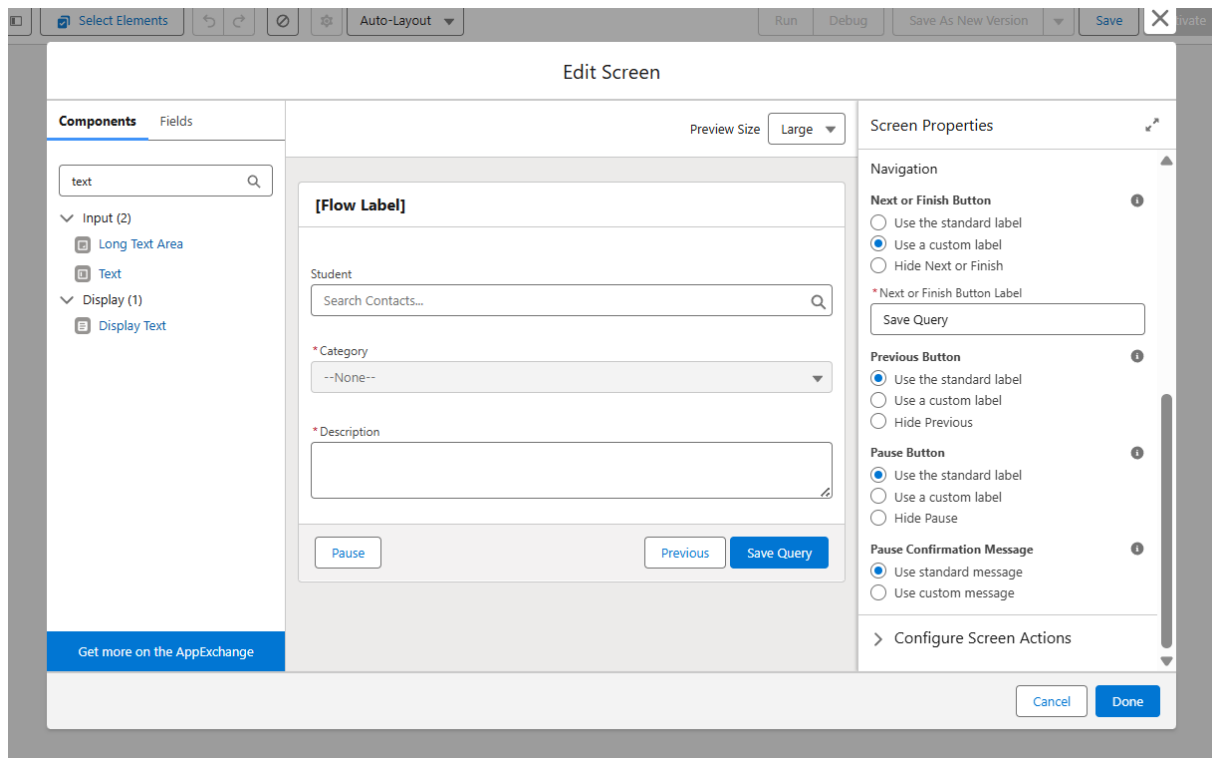
The screenshot shows the Lightning App Builder interface for the 'Company Record Page'. The main content area displays a record for 'Innovatech Solutions' with related lists for 'Job Postings (0)', 'Contacts (1)', 'Opportunities (0)', 'Cases (0)', and 'Notes & Attachments (0)'. A red box highlights the 'Job Postings (0)' and 'Contacts (1)' related lists. The right sidebar shows configuration options for the 'Related List - Single' component, including a search bar, a dropdown for 'Parent Record', and a dropdown for 'Related List Type'.

The screenshot shows the final 'Company Record Page' layout in the Salesforce org. The layout includes a header with navigation links, a left sidebar with components, and a main content area with related lists. The layout is clean and modern, with a focus on the 'Related' and 'Details' tabs. The 'Related' tab is active, showing a message 'We found no potential duplicates of this Company.' and a list of 'Contacts (1)' with details for 'Innovatech Solutions'. The 'Details' tab is also visible, showing 'Job Postings (0)', 'Contacts (1)', 'Opportunities (0)', 'Cases (0)', and 'Notes & Attachments (0)'. The right sidebar shows configuration options for the 'Related List - Single' component.

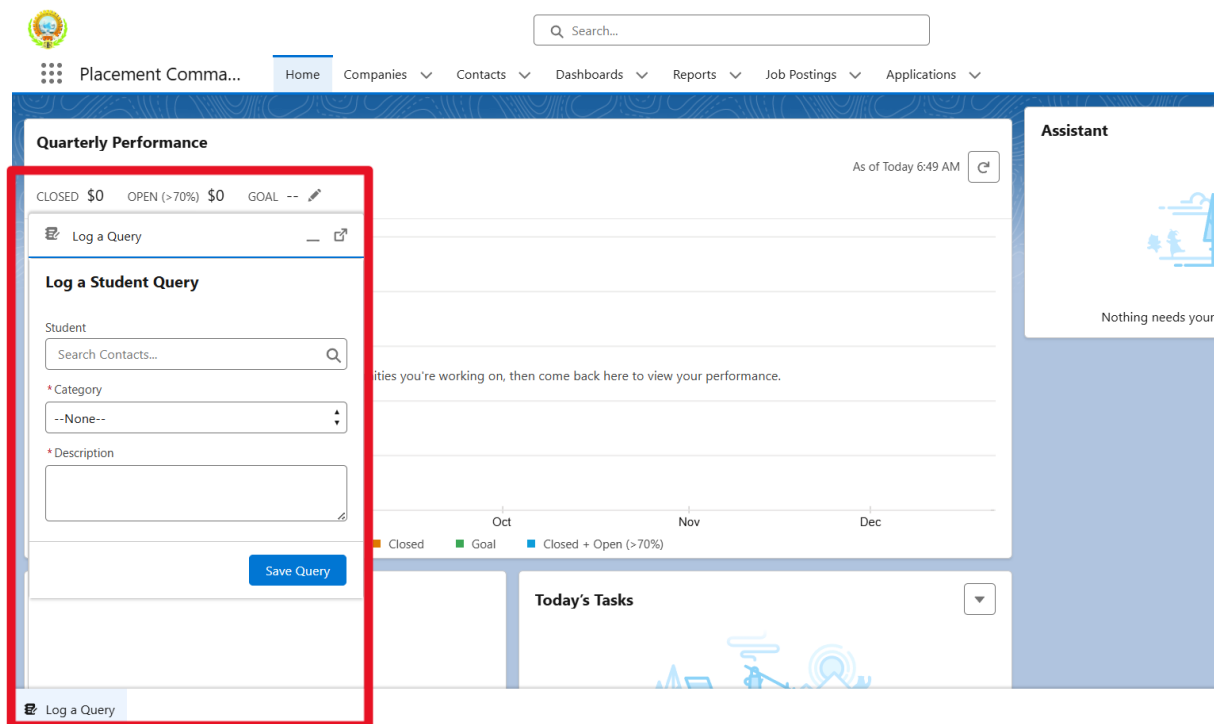
E. Utility Bar for Productivity: "Log a Query"

A quick-action tool was added to the app's footer, allowing users to perform a common task from anywhere.

- **Business Need:** T&P Officers are often on the phone with a student while navigating Salesforce. They need a way to log a student's query instantly without abandoning their current screen or task. A utility bar item provides this "always-on" functionality.
- **Implementation Steps:**
 1. A simple **Screen Flow** was built to act as the "pop-up" interface. The flow contains Lookup, Picklist, and Text Area screen components to capture the details of a new T&P Query. A Create Records element then saves this information.



2. This flow was then added to the "Placement Command Center" app via the App Manager > Utility Items section.
3. It was configured with a clear label ("Log a Query") and an intuitive icon, providing a persistent and easily accessible productivity tool in the app's footer.



3. Strategically Excluded Concepts & Justification

- **Lightning Web Components (LWC):** This project intentionally did not involve the creation of any custom Lightning Web Components. This includes all related concepts such as Apex with LWC, Wire Adapters, and Events.
 - **Justification:** The primary reason for this decision was efficiency and adherence to best practices. All of the UI requirements for this project were fully met using the standard, out-of-the-box components provided in the Lightning App Builder. Building a custom LWC would have required significant development time for

Phase 7: Integration & External Access Analysis Report

Date: September 23, 2025

1. Overview & Strategic Justification

This document serves as an analysis of the integration capabilities of the Salesforce platform and how they could be applied to the "Campus Recruitment Command Center" in the future.

A strategic decision was made not to implement a live external integration during this project phase. The core business requirements for the T&P Cell are focused on creating a

powerful, self-contained system to manage internal processes between officers and students. The focus was on getting that core part solid. Setting up big integrations just was not needed for this stage. So resources went into making things useful for the users.

Still, knowing how Salesforce links up with other systems matters a lot. It is key for the architecture. This report covers the main ways to do it. And shows we are set for adding more later.

2. Common Integration Patterns & Their Relevance to the Project

Salesforce can connect with external systems using several standard patterns. Here is how they could apply to our project:

- **API-Led Integration (Inbound):** This is when an external system needs to send data *into* Salesforce.
 - **Project Example:** The main university website could have a "Partner With Us" form for new companies. When a company submits that form, the website's backend could use a Salesforce **REST API** to automatically create a new Company record in our application. This would eliminate manual data entry for T&P officers.
- **Event-Driven Integration (Real-Time Sync):** This is a modern pattern where systems "notify" each other of changes instantly. It is event driven integration. It syncs things in real time. Systems ping each other right away on changes.
 - **Project Example:** The university's central Student Information System (SIS) is the master source for student data. When an administrator updates a student's official GPA in the SIS, it could publish a **Platform Event**. Our Salesforce application could be "listening" for this event and, upon receiving it, automatically update the GPA__c field on the corresponding Contact record in real-time.

3. Key Components Required for Integration

Successfully connecting Salesforce to another system requires a few key building blocks:

1. Web Services (REST/SOAP APIs):

- **What they are:** Think of these as secure "front doors" that allow other authorized programs to interact with our Salesforce data. Salesforce provides these automatically for all our objects, including Job Posting and Application.
- **Why they are needed:** They are essential for any inbound integration, like the website example above. The REST API is the modern standard because it's lightweight and easy to work with.

2. Authentication (OAuth 2.0):

- **What it is:** This is the secure "key" to the front door. Instead of sharing a username and password (which is insecure), an external application is given a secure access token via a **Connected App** in Salesforce.
- **Why it is needed:** Authentication is **mandatory** for any secure integration. It ensures that only approved applications can access our data, and that access can be revoked at any time without affecting user passwords.

3. Apex Callouts:

- **What they are:** This is the method used within Apex code when we need Salesforce to send a request *out* to an external system.
- **Why they are needed:** We would use a callout if a process *inside* our app needed to trigger something externally. For example, if we wanted a button to post a Job Posting to an external job board like LinkedIn. For performance reasons, these callouts must be **Asynchronous** (e.g., using `@future(callout=true)`), so the user doesn't have to wait for the external system to respond.

4. Named Credentials:

- **What they are:** This is a security feature inside Salesforce where we securely store the web address (URL) of an external system we trust.
- **Why they are needed:** This is a **mandatory** prerequisite for making an Apex Callout. It's a best practice because it separates the endpoint URL from the code, making the application more secure and easier to maintain.

4. A Potential High-Value Future Integration

While not part of the current scope, the most valuable future integration for this project would be a direct connection to the **University's Student Information System (SIS)**.

- **How it would work:** A nightly scheduled **Batch Apex** job would run. This job would make an API **Callout** to the SIS to get an updated list of all final-year students and their current GPAs. It would then automatically create or update the corresponding Contact records in Salesforce.
- **Benefits:** This integration would provide enormous value by:
 - Eliminating all manual data entry for student records.
 - Ensuring the GPA for every student is always accurate and official.
 - Providing a reliable and automated

Phase 8: Data Management & Deployment Final Report

Date: September 24, 2025

1. Overview

This phase focused on the operational duties of a Salesforce administrator: managing the application's data and understanding the deployment process. The primary goals were to ensure high-quality data, populate the system with realistic sample records for demonstration, and establish a data backup strategy. All tasks were completed using Salesforce's built-in, user-friendly tools.

2. Implemented Data Management Features

A. Initial Data Load using the Data Import Wizard

To bring the application to life, a set of sample data was loaded into the system.

- **Business Need:** A new application without data is difficult to test and impossible to demonstrate. Bulk-loading records is essential to create a realistic environment that showcases features like reporting, list views, and automation.
- **Implementation Steps:**
 1. Four CSV (spreadsheet) files were prepared with sample data for Companies (Accounts), Contacts (Students & Recruiters), Job Postings, and Applications.
 2. The **Data Import Wizard** was selected as the tool for this task. This tool is built directly into Salesforce Setup and is the ideal choice for its simplicity and effectiveness with this volume of data.

The screenshot shows the Salesforce Data Import Wizard interface. At the top, there is a progress bar with three steps: 'Choose data', 'Edit mapping', and 'Start import'. The 'Choose data' step is currently active. Below the progress bar, the title 'Import your Data into Salesforce' is displayed, followed by the note 'You can import up to 50,000 records at a time.' and a 'Help for this page' link. The main area is divided into three columns. The first column, 'What kind of data are you importing?', has tabs for 'Standard objects' and 'Custom objects'. Under 'Standard objects', 'Job Postings' is selected and highlighted with a red box. The second column, 'What do you want to do?', has options for 'Add new records' (selected), 'Match by', 'Which User field in your file designates record owners?', 'Which Company field in your file specifies the Master/Detail relationship?', and 'Trigger workflow rules and processes?'. The 'Add new records' option is highlighted with a red box. The third column, 'Where is your data located?', has a 'Drag CSV file here to upload' section. A 'CSV' file named 'Job_Postings.csv' is shown, with a 'Choose File' button and a 'Character Code' dropdown set to 'ISO-8859-1 (General US & Western European, ISO-LATIN-1)'. The 'Values Separated By' dropdown is set to 'Comma'. At the bottom right, there are 'Cancel', 'Previous', and 'Next' buttons.

3. A specific import order was followed to maintain the parent-child relationships between records: Companies were loaded first, then Contacts,

then Job Postings, and finally Applications. This ensured that every record was correctly linked to its parent.

Almost done

Choose data Edit mapping Start import

Edit Field Mapping: Job Postings

Your file has been auto-mapped to existing Salesforce fields, but you can edit the mappings if you wish. Unmapped fields will not be imported.

Edit	Mapped Salesforce Object	CSV Header	Example	Example	Example
Change	Job Posting Name	Name	Software Develop	AI Research Intern	Robotics Process Automation
Change	Company	Company__c	001gL000000chSk	001gL000000chSk	001gL000000chSmQAj
Change	Minimum GPA	Minimum_GPA__c	8.5	9.0	8.0
Change	Status	Status__c	Open	Open	Open
Change	Package (CTC)	Package_CTC__c	1800000	1200000	1500000
Change	Eligibility Branch	Eligibility_Branch__c	Computer Science	Computer Science	Mechanical/Electronics
Change	Location	Location__c	Bengaluru	Hyderabad	Pune

Cancel Previous Next

MAPPINGS TO CSV IMPORTED

B. Duplicate Rules to Ensure Data Quality

A system was configured to proactively prevent the creation of duplicate student records.

- **Business Need:** To maintain a reliable database, it is critical to prevent a single student from having multiple records. Duplicate records can split a student's application history and cause confusion for T&P officers.
- **Implementation Steps:**
 1. A **Matching Rule** was created on the Contact object. This rule tells Salesforce to consider a record a duplicate if it has the exact same First Name, Last Name, and Email as an existing record.

SETUP

Matching Rules

Rule Details

Object: Contact

Rule Name: Student_Matching_Rule

Unique Name: Student_Matching_Rule

Description:

Matching Criteria

Tell the rule which fields to compare and how.

Field	Matching Method	Match Blank Fields
First Name	Exact	☐
Last Name	Exact	☐
Email	Exact	☐
--None--	Exact	☐
--None--	Exact	☐

AND

AND

AND

AND

2. A **Duplicate Rule** was then activated. This rule uses the Matching Rule and is set to **Block** a user from saving a new record if it is identified as a duplicate. The user is shown a helpful alert message instead.

Rule Details

Rule Name:

Description:

Object: Contact

Record-Level Security: ☒ Enforce sharing rules ☐ Bypass sharing rules

Actions

Specify what happens when a user tries to save a duplicate record.

Action On Create:

Action On Edit: ☐ Alert ☐ Report

Alert Text:

Matching Rules

Define how duplicate records are identified.

Compare Contacts With:

Matching Rule:

Matching Criteria: (Contact: FirstName EXACT MatchBlank = FALSE) AND (Contact: LastName EXACT MatchBlank = FALSE) AND (Contact: Email EXACT MatchBlank = FALSE)

Field Mapping: ☒ Mapping Selected

3. Testing the Duplicate Rules

Similar Records Exist

This record looks like an existing record. Make sure to check any potential duplicate records before saving.

[View Duplicates](#)

C. Data Export for Backup and Recovery

An automated, weekly data backup schedule was established as a fundamental data protection measure.

- **Business Need:** The information managed by the T&P Cell is valuable. A regular backup process is a critical safety net that protects against accidental data deletion or corruption and ensures the data can be recovered if needed.
- **Implementation Steps:**

1. The native **Data Export** service within Salesforce Setup was used.
2. The service was configured to run automatically once a **Month** during off-peak hours. (2:00 am)
3. The export was set to include **all data** in the application, ensuring a complete and comprehensive backup is generated. The system administrator automatically receives an email with a link to download the backup files.

SETUP
Data Export

Schedule Data Export Help for this Page

Schedule Data Export Save Cancel

Export File Encoding: ISO-8859-1 (General US & Western European, ISO-LATIN-1) ▼

Include images, documents, and attachments: ☐ ?

Include Salesforce Files and Salesforce CRM Content document versions: ☐ ?

Replace carriage returns with spaces: ☒

Schedule Data Export

Frequency: ☐ On day 1 of every month ☒ On the 1st Sunday of every month

Start: 9/24/2025 [9/24/2025]

End: 10/9/2026 [9/24/2026]

Preferred Start Time: 2:00 AM ▼

Exact start time will depend on job queue activity.

Save Cancel

Exported Data
Select what type of information you would like to include in the export. The data types listed below are the Apex API names. If you are not familiar with these names, select Include all data for more options.

3. Strategically Excluded Concepts & Justification

- **Data Loader:** The separate Data Loader application was not used for this project's data tasks.
 - **Justification:** The **Data Import Wizard** was the more appropriate and efficient tool for the project's needs. Since the initial data load was small and involved standard objects, the wizard provided all necessary functionality without requiring an external application to be installed and configured. This choice demonstrates using the right tool for the job.
- **Deployment Tools (Change Sets, VS Code & SFDX):** The process of packaging and deploying the application to another Salesforce org was not performed.
 - **Justification:** This project was built entirely within a single, standalone **Developer Edition org**. Deployment tools are designed specifically for moving customizations between connected Salesforce environments (e.g., from a Sandbox to a Production org). As a Developer org is an isolated environment, there was no target org to deploy to, making this phase not applicable to the project's structure.

- **Packages (Managed/Unmanaged):** The application was not bundled into a formal package.
 - **Justification:** Packaging is a feature used to distribute an application for installation in other, unrelated Salesforce orgs (often via the AppExchange). The "Campus Recruitment Command Center" is a custom-built, internal tool intended only for use within this specific university. Therefore, creating

Phase 9: Reporting, Dashboards & Security Final Report

Date: September 25, 2025

1. Overview

This phase focused on translating the application's operational data into strategic business intelligence. The primary objective was to build analytical tools for the Head of Placements and T&P Officers to monitor progress, identify trends, and measure the success of placement activities. This was achieved by creating a powerful custom report type, building key reports, and visualizing them in a central, dynamic dashboard. The phase concluded with a final review of the application's security and administrative settings.

2. Implemented Analytics & Features

A. Custom Report Type: The Foundation for All Reporting

A custom report type was created to serve as the specific template required for all placement-related analysis.

- **Business Need:** To effectively analyze the placement pipeline, reports must be able to pull data simultaneously from Companies, their Job Postings, and the Applications submitted by students. A custom report type is the only way to create this specific, multi-object view.
- **Implementation Steps:**
 1. A new **Custom Report Type** was created with **Job Postings** as the primary object. This was a strategic choice to ensure all jobs could be reported on.

Report Type Focus

Primary Object Job Postings

Details

* Display Label: Job Postings with Applications and Company

* API Name: Job_Postings_with_Applications_and_Company

* Description: Shows all Job Postings and their related student applications.
Note: Description will be visible to users who create reports.

* Store in Category: Other Reports

Set Availability

An in-development report type is visible only to users with the Manage Custom Report Types permission. A deployed report type is available to all users.

Status

☐ In Development

☒ Deployed

Cancel Save

2. A relationship was established to the **Applications** child object. The relationship was configured as "Job Postings may or may not have Applications" to ensure all jobs are included in reports, even those without any applicants yet.

New Custom Report Type

2 Define Report Records Set

Select related objects to define which records are included in reports using this report type.

A Job Postings
Primary Object

B Applications

A to B Relationship:

☐ Each "A" record must have at least one related "B" record.

☒ "A" records may or may not have related "B" records.

(Click to relate another object)

Previous Cancel Save

3. Because Company is the parent object of Job Posting, its fields were automatically included, creating a complete and flexible three-object reporting structure.

Details

Display Label Job Postings with Applications and Company

API Name Job_Postings_with_Applications_and_Company

Description Shows all Job Postings and their related student applications.

Created By Pawan Rajak, 9/25/25, 7:02 PM

Store in Category other

Deployment Status Deployed

Modified By Pawan Rajak, 9/25/25, 7:02 PM

Object Relationships

Job Postings (A)

with or without related records from Applications (B)

Fields

Source Object	Included Fields
Job Postings	14
Applications	8

B. Core Reports: Answering Key Business Questions

Two essential reports were built using the new custom report type to provide immediate value.

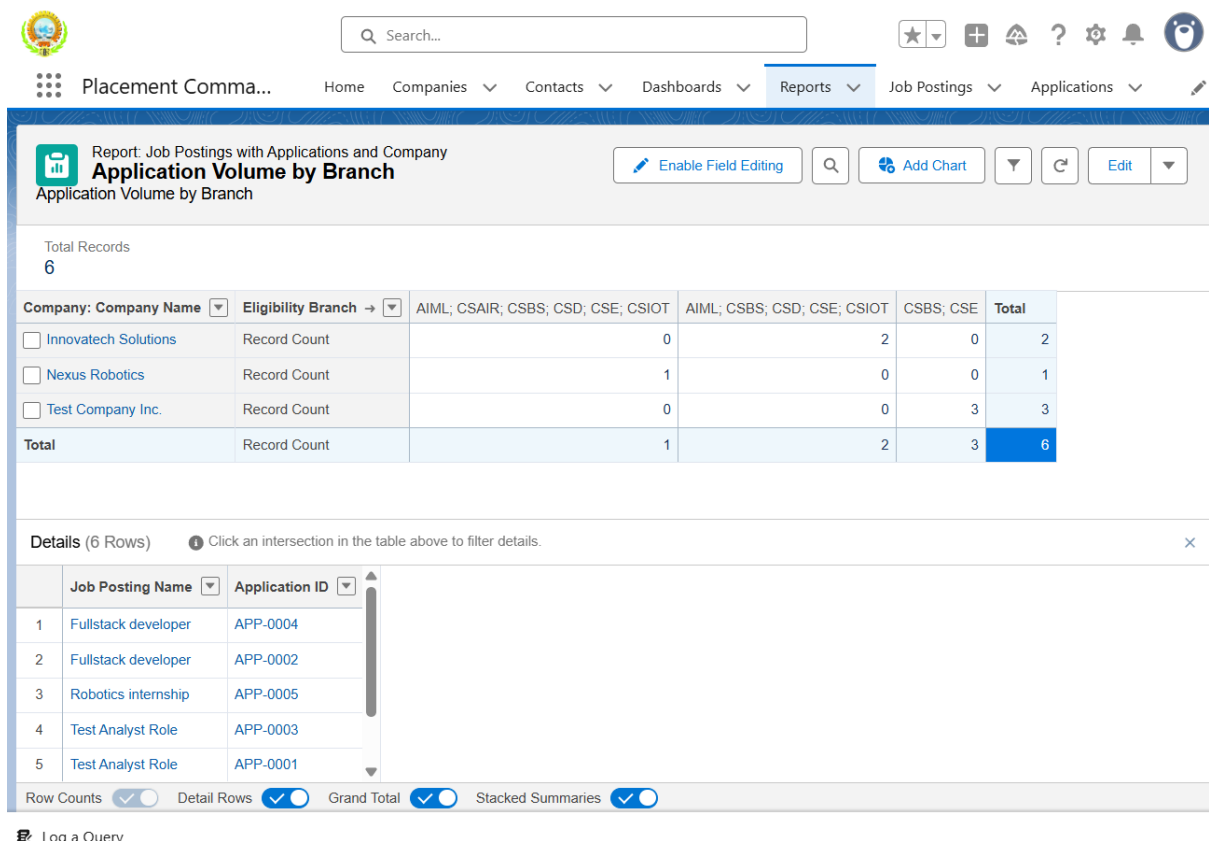
1. Placement Status by Company (Summary Report):

- **Purpose:** To give T&P Officers a clear, consolidated overview of the application pipeline for each company.
- **Type:** This is a **Summary Report**, as it groups data to provide subtotals and high-level analysis.
- **Configuration:** The report uses Company Name and Application Status as row groupings, creating a clear, hierarchical view of application progress for each corporate partner.

<

2. Application Volume by Branch (Matrix Report):

- **Purpose:** To identify which student branches are most actively applying to which companies. This helps the T&P cell understand engagement and target their outreach efforts.
- **Type:** This is a **Matrix Report**, as it groups data by both rows (Company Name) and columns (Eligibility Branch).
- **Configuration:** This cross-tabulated report provides a high-level summary of application trends between academic departments and companies.

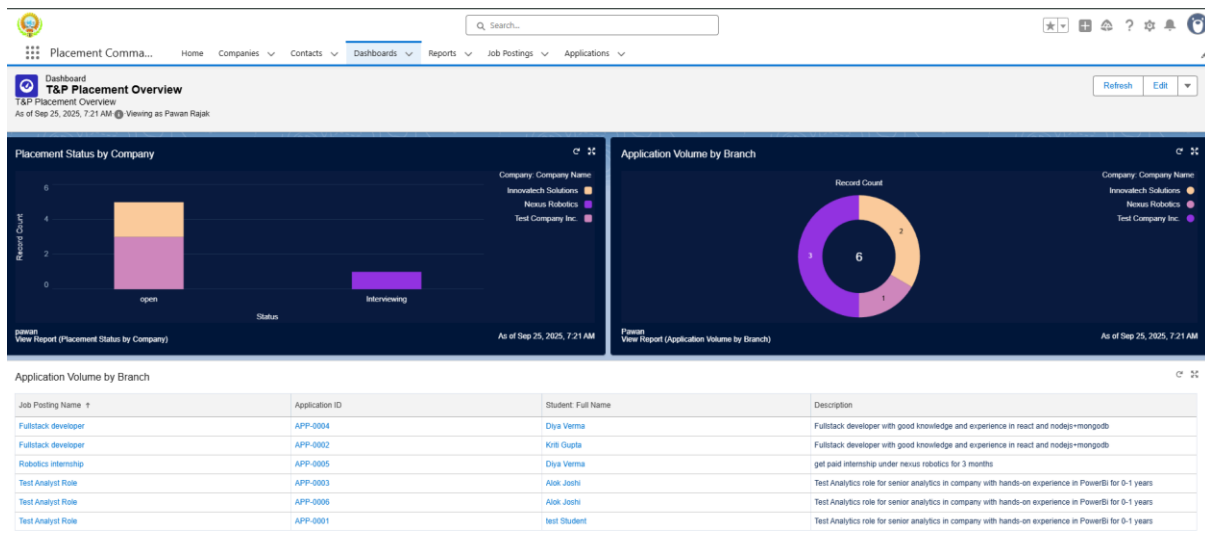


C. Dynamic Dashboard: The "T&P Placement Overview"

A central dashboard was created to serve as the main analytical command center for the Head of Placements.

- **Business Need:** Management requires a high-level, visual summary of key performance indicators without needing to run multiple individual reports. A dashboard consolidates this information into easily digestible charts and tables.
- **Implementation Steps:**
 1. A new **Dashboard** was created, titled "T&P Placement Overview".

- Components were added to visualize the two reports created above: a **Stacked Bar Chart** for placement status and a **Lightning Table** for the application volume matrix.
- Crucially, the dashboard was configured to be **Dynamic**. By setting it to be "Viewed as the dashboard viewer," the dashboard automatically respects Salesforce security rules, ensuring that users can only see the data they have permission to access.
- This new dashboard was then added to the **T&P Officer Home Page**, making it the first thing users see when they log into the application.



3. Security & Administrative Review

A final check was performed to ensure the application's settings are secure, auditable, and follow best practices.

- Sharing Settings (OWD):** Verified that the Application and T&P Query objects are set to **Private**, confirming the baseline security model is correctly configured to protect student data.
- Field-Level Security (FLS):** Verified that the visibility for key fields, such as Package (CTC) on the Job Posting object, is correctly configured for the Student and T&P Officer profiles as per the project requirements.
- Session Settings:** Confirmed that standard session timeout policies are in place via the Session Settings page, demonstrating an understanding of user session management.
- Setup Audit Trail:** Demonstrated the use of the **Setup Audit Trail** to track all administrative changes made to the org, confirming that all recent work (new reports, dashboards, etc.) was properly logged.

4. Strategically Excluded Concepts & Justification

- **Joined Reports:** This report type was not implemented.
 - **Justification:** The Custom Report Type created for this project provided a complete, unified view of all related data. Joined Reports are primarily used to display unrelated data in side-by-side blocks, a requirement which this project does not have due to its well-designed, relational data model. The Custom Report Type was the more powerful and appropriate tool for the business needs.
- **Login IP Ranges:** Specific IP ranges were not configured for user profiles.
 - **Justification:** In a real-world university setting, users like T&P Officers may need to access the system from various locations (home, campus, travel) with dynamic IP addresses. Restricting access to a specific IP range would be impractical and could lock out legitimate users. This security measure is best suited for a traditional corporate office environment with a static IP range.

Phase 10: Final Presentation & Handoff Report

Date: September 26, 2025

1. Pitch Presentation Summary

This section summarizes the core narrative of the project, as presented in the final demo.

A. The Problem Statement

University Training & Placement (T&P) cells manage a highly complex operation involving numerous corporate partners and thousands of student applications. The traditional reliance on disconnected tools like spreadsheets, manual emails, and shared documents creates significant challenges. This process is inefficient, prone to human error, results in a poor and confusing experience for students, and provides no real-time visibility for university leadership into one of their most critical success metrics: student placements.

B. The Solution & Its Impact

The "Campus Recruitment Command Center" is a centralized, intelligent application built on the Salesforce platform to solve these challenges. The solution provides a 360-degree view of all placement activities, from corporate relations to individual student applications, transforming a chaotic process into a streamlined, data-driven operation.

- **Impact for T&P Officers:** A massive increase in daily efficiency. By automating repetitive tasks like approvals, email alerts, and data validation, officers can save

hours of manual work and focus on high-value activities like building new corporate relationships.

- **Impact for Students:** A clear, simple, and transparent application process. Rules-based validation ensures they only apply for roles for which they are eligible, improving their experience and success rate.
- **Impact for Management:** For the first time, university leadership has access to real-time, reliable data through dynamic dashboards. This enables strategic decision-making based on live insights into which companies are hiring, which student branches are most successful, and the overall health of the placement pipeline.

2. Demo Walkthrough Summary

The final video demonstration showcased the end-to-end functionality of the application by following a core use case: posting a new job and processing applications as a single administrator. The key features demonstrated were:

- **Custom User Interface:** A branded "Placement Command Center" Lightning App with a custom, dashboard-driven Home Page and enhanced record pages that provide a 360-degree view of data.
- **Process Automation:** A multi-step Approval Process on the Job Posting object, complete with an automated, professionally formatted HTML Email Alert to notify the approver.
- **Custom Apex Logic:** A before insert Apex Trigger on the Application object that performs a complex, cross-object validation to check a student's GPA against the job's minimum requirement, successfully blocking ineligible applications with a custom error message.
- **Live Analytics:** A dynamic Dashboard that visualizes key reports and updates in real-time as new data is created, providing an instant feedback loop for management.

3. Handoff Documentation & Project Summary

This project represents a complete, end-to-end implementation of a custom Salesforce application. The key components developed are as follows:

- **Data Model:** Three core custom objects (Job Posting, Application, T&P Query) with secure master-detail and lookup relationships. Standard objects (Account, Contact) were customized with record types to support the business process.
- **Security Model:** A robust security model was implemented using custom Profiles for "T&P Officer" and "Student," with specific object and field-level security. The Organization-Wide Defaults were set to private where necessary to protect student data.

- **Automation:** A mix of Validation Rules, a multi-step Approval Process, and a Record-Triggered Flow were built using modern Salesforce tools to enforce business logic and automate tasks.
- **Custom Code:** A bulk-safe Apex Trigger was authored to handle complex validation logic, supported by a corresponding Apex Test Class with 100% code coverage.
- **User Interface:** A custom Lightning App, a dynamic Home Page, and an enhanced Record Page were all built declaratively using the Lightning App Builder to create an intuitive user experience.
- **Analytics:** A Custom Report Type was created to link all necessary objects. Two core reports (Summary and Matrix) and a dynamic Dashboard were built to provide comprehensive business intelligence.

A potential future enhancement would be to integrate this system with the university's central Student Information System (SIS) via a REST API to automate the creation and daily synchronization of student records.

4. LinkedIn/Portfolio Project Showcase Summary

The following is a concise summary suitable for a professional portfolio or LinkedIn profile:

- **Project: Campus Recruitment Command Center (Salesforce Platform)**
 - Designed and built a custom Salesforce application from the ground up to automate and streamline the end-to-end operations of a university's Training & Placement cell.
 - Developed a custom data model with three objects, configured distinct user profiles with granular field-level security, and implemented a multi-step approval process with HTML email alerts using Flow.
 - Authored a bulk-safe Apex Trigger to enforce complex, cross-object validation rules based on student GPA, ensuring data quality and adherence to business logic. Authored a corresponding test class with 100% code coverage.
 - Created a dynamic and intuitive user interface using the Lightning App Builder, including a custom Lightning App, a dashboard
 - Github: <https://github.com/pawan-kumar-rajak/Campus-Recruitment-Command-Center>
 - LinkedIn: <https://www.linkedin.com/in/pawan-kumar-rajak-2010b52b4>
 - Demo Video: <https://drive.google.com/file/d/16WipTPcA8AotpiI0-T4dvIK2MiCgED8D/view?usp=sharing>